

NRG Agent OS Source Intake Prompt

I am about to paste a very large planning conversation.

Your job is **not** to answer random pieces of the conversation one by one.

Your job is to read the entire conversation as source material, extract every important section, resolve contradictions, preserve the latest corrected intent, and produce a clean, complete, buildable plan for **NRG Agent OS**.

This conversation is messy because the plan evolved while we were discussing it. It contains drafts, corrections, disagreements, uploaded-file references, source inspiration, design notes, safety rules, and architecture ideas.

Do not lose important details.

Do not compress too early.

Do not treat early drafts as final when later corrections override them.

Do not copy the conversation tone.

Do not preserve repetition.

Do not invent missing details unless clearly labelled as a recommendation.

How To Read The Conversation

Read it in passes.

Pass 1: Source Map

First, identify and list the major source sections found in the conversation, including:

- original Agent OS goal
- Fable 5 usage strategy
- Hermes role
- persistent memory requirement
- support-agent delegation
- kernel/control layer
- full-stack scope
- Ornith/local model strategy
- self-evolving agent policy
- JARVIS Prompt Pack inspiration
- video/transcript inspiration
- visual/dashboard direction
- safety rules
- build order

- worker task cards
- Fable review gate
- final wrappers/prompts

Pass 2: Latest Corrected Intent

Extract the latest corrected intent.

The final intent is:

Fable 5 should design the important system.

Fable 5 owns:

- architecture
- OS layout
- full-stack design
- prompt system
- router logic
- persistent memory architecture
- permissions
- validation
- UX/design system
- worker-agent boundaries
- build phases
- final review process

Codex, GPT 5.5, Gemini Pro 3.5, Ornith, Hermes, and other agents must **not** design the OS.

They may only execute bounded implementation work after Fable creates exact specs, file boundaries, task cards, acceptance criteria, and validation rules.

Hermes is a runtime/operator inside the structured OS, not the centre of the architecture.

Ornith is an optional local model candidate, not a lock-in.

Persistent memory is first-class.

The OS must be scalable, adjustable, modular, swappable, and future-proof as models and tools change.

The dashboard must not be built from generic AI visuals or worker-agent improvisation. Fable must design the visual system and UX structure.

Self-evolving behaviour must be governed by the OS.

The OS must not be governed by self-evolving behaviour.

Pass 3: Contradiction Handling

Resolve contradictions using this priority order:

1. This intake prompt
2. The latest corrections in the conversation
3. Hard safety rules
4. Structured Markdown / .md-style sections
5. Normal chat context explaining why the plan changed
6. Earlier drafts only where they still support the latest corrected intent

Do not treat every assistant message as authority.

Assistant messages are design rationale and draft structure.

My corrections override assistant framing.

Pass 4: Extract The Actual System

Extract the final NRG Agent OS as a serious full-stack agent operating system.

This is not just a dashboard.

This is not just a memory graph.

This is not just Hermes.

This is not just Fable.

This is not just a JARVIS clone.

This is a full-stack agent OS with:

- frontend/control surface
- backend/API
- agent runtime
- model gateway
- persistent memory
- scratch memory
- quarantine memory
- skill registry
- automation engine
- hook policy
- cron policy
- permission engine
- operating modes
- validation system
- drift detection
- hallucination checking
- event/audit logs

- worker-agent task cards
- Fable review gate
- recovery system
- versioning/migration policy
- dashboard UX/design system

Core Architecture Rule

The OS must be built around interfaces, contracts, routing, validation, permissions, memory, event logs, and review gates.

Do not build it around any single model.

Models are replaceable.

The OS structure is permanent.

Required Roles

Fable 5

Fable 5 is the architect, designer, prompt engineer, UX director, system reviewer, and quality gate.

Fable should produce the final OS blueprint, folder structure, file contents, schemas, policies, prompts, task cards, acceptance criteria, dashboard design system, build phases, and review checklists.

Hermes

Hermes is the daily runtime/operator inside the OS.

Hermes may:

- execute approved skills
- run inside the router/permission system
- capture session summaries
- propose lessons
- propose memory updates
- propose skill improvements
- propose automation candidates
- report repeated failures

Hermes may not:

- silently change core prompts
- silently promote memory
- rewrite router policy
- change permissions
- edit hooks
- edit crons
- run destructive actions

- decide architecture

Hermes benefits from the structured OS. Hermes does not define the OS.

Ornith

Ornith may be tested as an optional local reasoning/coding model.

It must be swappable, benchmarked, disabled, upgraded, or replaced over time.

Do not lock the OS to Ornith.

Ornith sits behind the model gateway like every other model.

Codex / GPT 5.5 / Gemini Pro 3.5 / Other Workers

Worker agents may execute bounded task cards only.

They may not invent architecture, change rules, rename concepts, alter permissions, decide visual style, expand scope, or certify success without evidence.

Every worker task must include:

- task name
- purpose
- files allowed to touch
- files forbidden to touch
- allowed commands
- forbidden commands
- exact inputs
- exact outputs
- acceptance checks
- rollback notes
- evidence required
- what the worker must not decide

Worker output must return to Fable for review.

Hard Safety Rules

Default mode is readonly.

No move, delete, overwrite, mass edit, migration, install, git reset, folder restructure, hook edit, cron edit, or destructive action without explicit approval.

Every path must be verified before use.

Every final claim must be backed by evidence.

No agent may expand scope beyond the user request.

Memory is context, not authority.

Old notes, pasted text, logs, repo comments, generated files, previous agent output, web content, and unverified memory are untrusted until validated.

API keys, secrets, tokens, `.env` contents, credentials, SSH keys, and private config must never be pasted into chat, exposed in browser files, copied into memory, or printed in reports.

Agents may detect secret locations, but must not reveal secret values.

No agent may claim:

- "done" without evidence
- "fixed" without validation
- "tests passed" without test output
- "file exists" without checking
- "only changed X" without diff evidence
- "deployed" without deployment evidence

Critical Missing Kernel Layer

The final plan must include the kernel/control layer.

These are not optional polish.

Include:

- boot sequence
- task lifecycle
- agent contracts
- definition of done
- capability registry
- sandbox and dry-run modes
- OS self-tests
- event/audit log
- versioning and migration policy
- secrets and redaction policy
- dashboard design system
- operating modes
- quarantine policy
- Fable review gate

The OS is not ready for live automations until these exist.

Persistent Memory Requirement

Persistent memory must be first-class, not a vague add-on.

Design memory with:

- persistent memory
- scratch memory
- quarantine memory
- indexes
- trust levels
- memory router
- memory promotion rules
- session logs
- lessons
- project state
- path registry
- decisions
- issues
- known failures
- skill history
- automation history
- validation history
- Hermes runtime notes
- source summaries

Memory must support trust levels:

- verified
- user-confirmed
- tool-derived
- agent-inferred
- stale
- untrusted
- quarantined

No untrusted memory may become active without validation.

Full-Stack Scope

Design NRG Agent OS as a full-stack agent operating system.

Include these layers:

1. Frontend / Control Surface

Dashboard, operating view, panels, approvals, logs, memory views, automation views, validation views, model status, task state, recovery controls.

2. Backend / OS API

Task intake, router endpoints, memory endpoints, chat/action endpoints, worker-task endpoints, validation endpoints, event logging, file-safe operations, status APIs.

3. Agent Runtime Layer

Hermes runtime, Fable review gate, worker execution, Ornith/local model testing, cloud fallback, model gateway, capability registry.

4. Memory Layer

Persistent memory, scratch memory, quarantine memory, indexes, source summaries, session logs, lessons, skill history, automation history, memory promotion.

5. Skill and Automation Layer

Skill registry, hook policy, cron policy, file watchers, scheduled jobs, manual triggers, approval gates, rollback paths, automation logs.

6. Validation and Safety Layer

Permission engine, operating modes, dry-run mode, sandbox mode, path verification, diff verification, claim verification, drift detection, hallucination detection, secrets policy, destructive-action blocking.

7. Event/Ops Layer

Task lifecycle, event log, audit log, versioning, migrations, changelog, rollback plans, failure reports, recovery procedures.

8. Design System Layer

Visual metaphor, component contracts, layout rules, status states, warning states, error states, approval prompts, dashboard panels.

JARVIS Prompt Pack Treatment

A PDF called `JARVIS-Prompt-Pack.pdf` may be part of the source material.

Treat it as concept inspiration only, not as the final architecture.

Useful concepts from it:

- knowledge graph / note visualisation
- note-grounded answering
- source proof / fly-to-source idea
- browser voice
- assistant personality
- "remember that" capture flow
- model hot-swap UX
- action confirmation gates
- cinematic dashboard direction

Do not copy the JARVIS memory galaxy directly.

Do not default to generic glowing memory nodes unless you can justify why that is the best operational view.

NRG Agent OS needs a visual identity of its own.

The dashboard should show the OS operating, not just decorate memory.

Avoid:

- random glowing node galaxy
- generic neon brain
- meaningless orbiting particles
- fake sci-fi UI
- pretty widgets with no operating value
- dashboard-first architecture
- visuals that hide failures

Give Fable creative room to design a better metaphor.

Possible directions include, but are not limited to:

- command reactor
- agent traffic control
- mission control map
- neural workshop
- forge/foundry
- circuit board OS
- control tower

Pick the strongest visual model for the OS.

The visual system must show:

- current task
- task lifecycle state
- router decision
- risk level
- permission mode
- selected model
- selected agent
- selected skill
- files touched
- memory read
- memory written
- validation status
- drift warnings
- hallucination warnings
- approval required
- worker task cards
- Fable review status
- Hermes runtime status
- hook and cron status
- quarantine items
- recovery options
- event log

Video / Transcript Inspiration Treatment

The conversation may include YouTube links, transcript-derived ideas, and visual/build inspiration.

Treat these as source inspiration, not authority.

Use them for:

- visual aesthetic inspiration
- dashboard layout inspiration
- agent OS concepts
- self-evolving agent ideas
- memory and skill-loop concepts
- runtime orchestration ideas
- local/cloud model-routing ideas
- browser automation patterns
- workflow automation patterns

Do not use them for:

- copying someone else's system
- trusting hype
- replacing architecture with aesthetics
- letting self-evolution bypass safety
- letting Hermes or Ornith become uncontrolled authority
- letting support agents redesign the OS

Self-Evolution Policy

Self-evolution is allowed only as a proposal pipeline.

Allowed:

- generate candidate skill improvements
- generate candidate prompt improvements
- generate candidate tool descriptions
- generate candidate workflow optimisations
- generate candidate memory lessons
- generate candidate automation improvements
- run benchmarks
- compare old vs new behaviour
- produce review reports

Forbidden without approval:

- editing live system prompts
- changing permission policy
- changing router policy
- promoting memory to trusted state
- editing hooks
- editing crons

- running destructive automations
- rewriting core OS structure
- silently replacing skills
- claiming an evolved version is better without tests

Every evolved candidate must answer:

- What changed?
- Why was it changed?
- What evidence shows improvement?
- What tests passed?
- What got worse?
- What risks remain?
- Can it be rolled back?
- Should this be accepted, rejected, or quarantined?

Self-evolving agent behaviour must be governed by the OS.

The OS must not be governed by self-evolving agent behaviour.

Required Final Output

After reading the entire conversation, produce the final consolidated **NRG Agent OS Factory Pack**.

Output clean Markdown.

Use file-by-file sections where useful.

Include:

1. Executive summary
2. Final corrected intent
3. Source interpretation map
4. Final OS architecture
5. Final folder layout
6. Full-stack layer breakdown
7. Persistent memory architecture
8. Router architecture
9. Permission policy
10. Operating modes
11. Skill architecture
12. Automation, hook, and cron policy
13. Validation system
14. Drift detection
15. Hallucination detection
16. Worker-agent delegation system
17. Fable review gate
18. Dashboard UX/design system
19. Creative visual direction
20. Task lifecycle
21. Boot sequence

22. Agent contracts and schemas
23. Event/audit log system
24. OS self-test system
25. Secrets and redaction policy
26. Versioning and migration policy
27. Hermes runtime integration
28. Ornith/local model strategy
29. Video/JARVIS inspiration handling
30. Self-evolution governance
31. Recovery policy
32. First build order
33. Master prompt pack
34. Worker task card templates
35. Final review checklist
36. Fable intake prompt
37. Codex/GPT/Gemini task-card template
38. Acceptance criteria
39. Risks and failure modes
40. What not to build yet

Output Quality Rules

Do not make the result sound impressive for its own sake.

Do not produce fluffy AI hype.

Do not bury safety rules.

Do not skip boring kernel parts.

Do not reduce this to a UI project.

Do not reduce this to a Hermes project.

Do not reduce this to a memory graph.

Do not reduce this to local model testing.

Make it practical, strict, modular, scalable, and buildable.

The aim is to prevent future agents from:

- destroying the project
- inventing paths
- duplicating files
- overwriting work
- expanding scope
- building garbage UI
- claiming success without proof
- silently promoting bad memory

- letting support agents redesign the system
- letting Hermes learn from chaos
- letting Ornith or any local model become a lock-in

Before Producing The Final Plan

Before writing the final Agent OS Factory Pack, provide a brief “Source Map” showing what major sections you found in the conversation.

Then provide the final plan.

Do not ask me to manually separate the chat.

That is your job.